

Report-石亦烜

人工智能导论——四子棋实验报告

计41 石亦烜 2023010302

1 算法思路

1.1 算法流程

本次实验我采用的是UCT算法，即结合了MCTS和UCB公式的博弈树搜索算法。算法基本框架如下：

```

function UCT(root, timeLimit):
    while now_time < timeLimit:
        node ← Select(root)
        result ← Simulate(node)
        Backup(node, result)
    return BestChild(root)

function Select(node):
    while node is fully expanded and not terminal:
        node ← BestChild(node, UCTValue)
    expanded_node = expand(node);
    return expanded_node

function Simulate(node):
    while not terminal(node):
        action ← RandomAction(node)
        node ← ApplyAction(node, action)
    return GameResult(node)

function BestChild(node, valueFunction):
    bestValue ← - ∞
    bestChild ← null
    for each child in node.children:
        if valueFunction(child) > bestValue:
            bestValue ← valueFunction(child)
            bestChild ← child
    return bestChild

function UCTValue(node):
    if node.visitCount == 0:
        return ∞
    return (node.winCount / node.visitCount) + C *
sqrt(log(node.parent.visitCount) / node.visitCount)

```

具体来说，每次树搜索过程包括选择、扩展、模拟、回溯四步。

- 选择(Select): 对于已经不能再扩展的节点，根据UCB公式选择其最优子节点，即在保留一定探索性的条件下选择最好的子节点。直到找到一个能够扩展的节点。
- 扩展(Expand): 对选择中找到的结点进行扩展。在实际的代码中，选择和扩展均在 `function select()` 中实现
- 模拟(Simulate): 利用蒙特卡洛方法对该节点进行随机模拟，知道分出胜负，返回结果。若机器胜则返回1，用户胜则返回-1，平局返回0。
- 回溯(Backup): 沿刚才的搜索路径更新路径上所有节点的 `win_count` 和 `visit_count`，注意更新时要考虑当前结点的落子方时机器还是用户。

1.2 具体实现

在原有的实现框架上，我新增了 `MCTS.h`，`MCTS.cpp`，`Node.h`，`Node.cpp` 四个文件，具体说明如下。

1. `MCTS.cpp`: 存储蒙特卡洛搜索树，主要成员包括根节点 `root`，棋盘状态 `mct_board`，`mct_top`，用于在搜索过程中及时更新棋盘状态。封装了最终输出落子点的 `getBestMove` 方法，以及上述选择、模拟、回溯方法。
2. `Node.cpp`: 树节点类，主要记录每个结点对应的落子，落子方以及棋盘状态 `node_top`。封装的方法包括对节点进行扩展的方法 `expand()`，和计算UCB值方法。

2 实验结果

在Saiblo与2~100号AI批量测试的结果如下：

批量测试 #74120

[我的批量测试](#)

97 3 0 100 100 97%

胜

负

平

已测评局数

总局数

胜率

被测试 AI



对应批量测试序号为#74120，网址为[批量测试 #74120 - Saiblo](#)

3 总结

这次实验我亲手实现了一次UCT信心上限树算法，对蒙特卡洛树搜索的过程有了更完整的理解。我也意识到了很多之前没有注意到的算法细节，比如回溯时对不同落子方的不同处理，对提前结束点（叶子结点）的处理等等，这些都是在debug的过

程中发现的错误。

最终，我实现的AI在与测试AI的对抗过程中取得了还不错的结果。

未来优化的方向应该在于引入必胜必败策略，提前对搜索树进行剪枝，以优化搜索效率。